

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 4. Operators and Expressions

Lecture: 30. Challenges using Expression

Section 1: Lecture Summary

This lecture focuses on practicing the writing of Python expressions by converting known mathematical formulas into Python code. The main content involves writing Python programs to compute areas of different geometric shapes (triangle, trapezium, circle), converting kilometers to miles, and calculating displacement using given physics formulas. The instructor explains each formula and demonstrates how to express it correctly in Python, including the use of variables, arithmetic operators, and importing modules (like `math` for the constant π). Students are encouraged to type and run these programs themselves using tools such as Jupyter Notebook or PyCharm and also complete provided coding exercises.

Section 2: Key Concepts and Explanations

- Expressions and Variables:

Expressions combine variables and mathematical operators to calculate values which are then stored in variables.

- Arithmetic Operators in Python:

- ```` for multiplication
- ``/`` for division (floating point division in Python 3)
- Parentheses `()` to enforce order of operations

- Area Calculations:

- Triangle Area:

Formula is $\left(\frac{1}{2} \right) \times \text{base} \times \text{height}$.

Python equivalent:

```
area = (base * height) / 2
```

Note: `1/2` in Python 3 results in `0.5` as a float (not integer division).

- Trapezium Area:

Formula is $\left(\frac{1}{2} \right) \times (a + b) \times h$, where `a` and `b` are the lengths of the parallel sides and `h` is the height.

Python equivalent:

```
area = 0.5 * (a + b) * h
```

- Circle Area:

Formula is πr^2 , where `r` is the radius and π is a constant (~ 3.14159).

Python uses the `math` module's constant:

```
import math
area = math.pi * r * r
```

- **Unit Conversion:**

- Kilometers to Miles:

Formula uses the conversion factor $1 \text{ km} = 0.621371 \text{ miles}$.

Python equivalent:

```
miles = 0.621371 * km
```

- **Displacement Calculation (Physics):**

- Formula:

$$d = \frac{v^2 - u^2}{2a}$$

where `v` is final velocity, `u` is initial velocity, and `a` is acceleration.

- Expressions must use parentheses carefully to maintain correct order:

$$d = (v * v - u * u) / (2 * a)$$

Important to avoid operator precedence mistakes.

- Taking User Input:

Though not explicitly coded here, students are advised to take base, height, radius, kilometers, velocities, and acceleration as user inputs and convert them to appropriate numeric types (`int` or `float`) before computation.

Section 3: Example Code and Use Cases

1. Area of a Triangle

```
b = 10    # base
h = 5     # height
area = (b * h) / 2
print('Area of Triangle:', area)
```

Calculates the area of a triangle with base 10 and height 5.

2. Area of a Trapezium

```
a = 4     # side a (parallel side)
b = 10    # side b (parallel side)
h = 5     # height
area = 0.5 * (a + b) * h
print('Area of Trapezium:', area)
```

Computes the area of a trapezium with given sides and height.

3. Area of a Circle

```
import math

r = 10 # radius
area = math.pi * r * r
print('Area of Circle:', area)
```

Calculates the area of a circle using the radius 10 and the π constant from the math module.

4. Kilometers to Miles Conversion

```
km = 15
miles = 0.621371 * km
print('Miles:', miles)
```

Converts 15 kilometers to miles using the standard conversion factor.

5. Displacement Calculation

```
v = 20 # final velocity
u = 10 # initial velocity
a = 2 # acceleration
d = (v * v - u * u) / (2 * a)
print('Displacement:', d)
```

Calculates the displacement from velocities and acceleration, demonstrating careful use of parentheses.

Section 4: Key Takeaways

- Writing mathematical expressions correctly in Python requires understanding operator precedence and use of parentheses to avoid errors.
- Python 3 uses floating-point division by default, so expressions like `1/2` produce `0.5`.

- Use the ``math`` module for mathematical constants like π instead of hardcoding numerical values.

- Converting known formulas into Python programs helps reinforce variable usage, arithmetic operations, and program structure.

- Taking input from users (not shown but recommended) and converting them to numeric types makes the programs interactive and dynamic.

- Small challenges like area calculations, unit conversions, and physics formulas are effective for practicing Python expression writing.

- Testing your programs in environments like Jupyter Notebook, PyCharm, or VSCode is encouraged to build hands-on skills.